

Data Handling and Infrastructure for AI

Data Lakes and Vector Databases

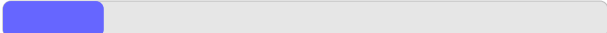
Tamás Grósz

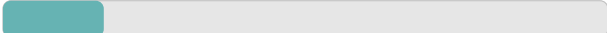
SETU

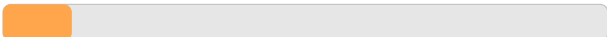
2026

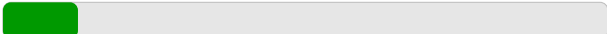


Workload

Lectures  4 / 24 hrs

Labs  4 / 24 hrs

Self-Study  23 / 202 hrs

Overall Progress  31 / 250 hrs

Where We Are

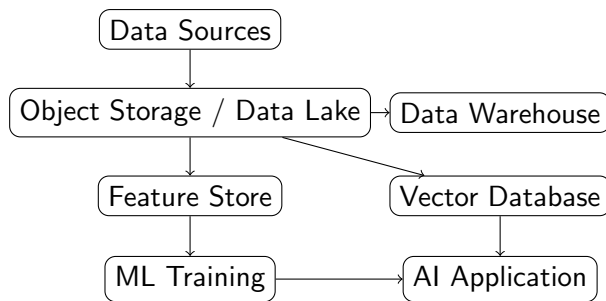
In the previous lecture we looked at:

- Batch vs. streaming processing
- Train / Dev / Test and Cross-Validation
- Data leakage
- I/O bottlenecks
- Sharding and the many-small-files problem
- SQL vs. NoSQL databases
- Distributed training and DDP

Today we broaden the picture:

Where does all the AI data actually live?

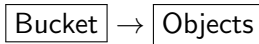
A Modern AI Data Architecture



There is rarely one “database” containing everything.

Object Storage

Object storage stores data as objects rather than traditional files.



`bucket/data/images/00001.jpg`

`bucket/data/train/part-0001.parquet`

Important properties:

- Extremely large capacity
- High durability
- Designed for distributed access
- Relatively inexpensive
- Accessible from many compute services

Cloud Object Storage

The three major cloud providers offer very similar concepts.

Cloud	Object Storage	Typical naming
Google Cloud	Google Cloud Storage (GCS)	Buckets / Objects
AWS	Amazon S3	Buckets / Objects
Microsoft Azure	Blob Storage	Storage Accounts / Containers

All provide:

- Large-scale storage
- APIs for programmatic access
- Access control
- Different storage classes / tiers
- Integration with cloud compute and AI services

GCS is Google Cloud's object storage service.

Typical structure:

```
gs://setu-dh-ai/
```

```
raw/  
processed/  
features/  
models/  
logs/
```

Useful for:

- Raw datasets
- Parquet / TFRecord / sharded datasets
- Images, audio and video
- Training data
- Model artefacts
- Data pipelines

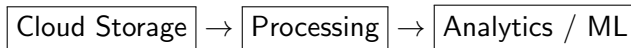
AWS: Amazon S3

- Object storage using buckets
- Common foundation for AWS data lakes
- Integrates with services such as Glue, Athena and SageMaker

Azure Blob Storage

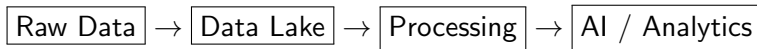
- Object storage using containers
- Common foundation for Azure data lakes
- Integrates with Azure Data Factory, Synapse and Azure ML

The architecture is conceptually similar:



What Is a Data Lake?

A **data lake** is a central repository for storing large amounts of data in its original or near-original form.



A lake may contain:

- CSV and Parquet files
- Images
- Audio and video
- JSON events
- Logs
- Database extracts
- ML datasets and model artefacts

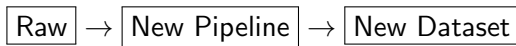
Data Lake: Why Keep Raw Data?

Suppose we transform:

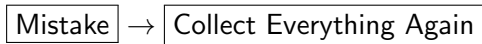
Raw data → Cleaned data → Features

What happens if we discover that our cleaning step was wrong?

If raw data is retained:



Without the raw data:



Principle

Keep an immutable or carefully versioned copy of important raw data.

The Data Lake Problem

A data lake can become a:

Data Swamp

if nobody knows:

- What a dataset contains
- Where it came from
- Who created it
- When it was collected
- Which version is being used
- Whether it is trustworthy
- Which transformations have been applied

Storage alone does not create useful data infrastructure.

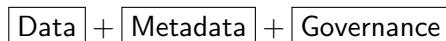
Data Catalogues and Metadata

A **data catalogue** provides information *about* datasets.

Example:

Dataset	Customer Transactions
Owner	Data Engineering Team
Created	2026-02-10
Rows	84,200,000
Format	Parquet
Update frequency	Daily
PII	Yes
Quality status	Validated

Metadata allows people and systems to discover and understand data.



Data Lakehouse

A **lakehouse** attempts to combine properties of:

Data Lake + Data Warehouse

The idea:

- Keep inexpensive object storage underneath.
- Add tables, schemas and transactional guarantees.
- Support both analytics and ML workloads.
- Provide better data management than a collection of files.

Examples of technologies in this ecosystem include:

- Apache Iceberg
- Delta Lake
- Apache Hudi

Data Warehouse vs. Data Lake

	Data Warehouse	Data Lake
Data	Structured	Structured + unstructured
Schema	Usually predefined	Often schema-on-read
Primary use	Analytics / BI	Data science / ML / storage
Typical storage	Managed tables	Object storage
Users	Analysts	Engineers / scientists / ML

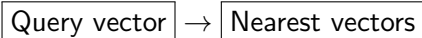
Modern systems increasingly blur this distinction.

Vector Databases

Modern AI applications often represent objects as vectors.

$$x = \begin{bmatrix} 0.12 \\ -0.83 \\ 0.44 \\ \vdots \\ 0.21 \end{bmatrix} \in \mathbb{R}^{1536}$$

A vector database stores these representations and supports **similarity search**.



This is particularly important for:

- Semantic search
- Recommendation
- Image search
- Retrieval-Augmented Generation (RAG)

Why Not Just Use SQL?

Suppose we have one million document embeddings.

A traditional query asks:

WHERE country = 'Ireland'

A vector query asks: Find vectors most similar to q , using a distance such as:

$$d(x, y) = \|x - y\|_2$$

or cosine similarity:

$$\text{sim}(x, y) = \frac{x^\top y}{\|x\| \|y\|}$$

Finding exact nearest neighbours can be expensive at large scale.

Vector databases therefore use specialized indexing methods.

Approximate Nearest Neighbour Search

For millions or billions of vectors, exact search may be too expensive.
Instead, systems use:

Approximate Nearest Neighbour (ANN)

The goal is:

Much faster search with small loss in exactness

Common approaches include:

- HNSW
- IVF
- Product Quantization
- Vector indexes integrated into databases

The trade-off becomes:

Vector Search + Metadata

Vectors are rarely useful on their own.

Imagine a document collection:

Document = (embedding, text, author, date, permissions)

A query might be:

Find documents similar to this question

but only:

from 2025, written by team X, that the user is allowed to see.

Therefore vector systems often combine:

Vector similarity + Metadata filtering

A typical Retrieval-Augmented Generation system:

Documents → Chunks → Embeddings → Vector DB

User Query → Embedding → Similarity Search

→ Retrieved Context → LLM → Answer

The vector database is therefore part of the **AI application's data infrastructure**.

Feature Stores

A **feature store** manages features used by ML models.

Example:

Customer	Features
123	age, transaction_count, avg_spend
456	age, transaction_count, avg_spend
789	age, transaction_count, avg_spend

A feature store can provide:

- Feature definitions
- Consistent transformations
- Historical feature values
- Training / serving consistency
- Online low-latency feature retrieval

The Training–Serving Skew Problem

Suppose training computes:

average purchase amount

using one implementation.

But production inference computes it differently.

Then:

Training features $\neq$ Production features

This is called **training-serving skew**.

A feature store can help ensure:

Same feature definition $\rightarrow$ Training + Inference

Data Versioning

Data changes.

$$D_1 \rightarrow D_2 \rightarrow D_3 \rightarrow \dots$$

If a model was trained on D_1 , we need to know:

- Which exact data was used?
- Which preprocessing code?
- Which feature definitions?
- Which model version?
- Which hyperparameters?

This creates the concept of:

Data lineage

Reproducibility

Consider:

Raw data $\rightarrow$ Cleaning $\rightarrow$ Features $\rightarrow$ Training $\rightarrow$ Model

Lineage records these relationships.

For example:

$$M_{42} \leftarrow D_{17} \leftarrow P_8 \leftarrow D_{\text{raw},12}$$

where:

- M_{42} = model version
- D_{17} = training dataset
- P_8 = preprocessing pipeline
- $D_{\text{raw},12}$ = raw data version

Storage Tiers

Not every piece of data needs the same performance.

Tier	Access	Example
Hot	Frequent	Current training data
Warm	Occasional	Recent datasets
Cold	Rare	Historical data
Archive	Very rare	Long-term retention

Cloud providers offer different storage classes to reflect these different access patterns.

Infrastructure principle

Don't optimize only for storage capacity. Optimize for the complete cost of storing *and* *accessing* data.

Data Compression

Compression can reduce:

Storage cost

and:

I/O volume

But compression also costs CPU time.

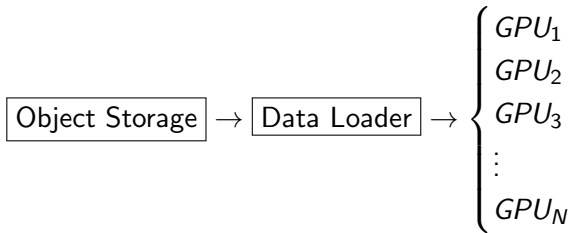
Uncompressed → Compression → Storage → Decompression → GPU

The optimal choice depends on:

- Compression ratio
- CPU cost
- Storage bandwidth
- Network bandwidth

Storage and AI Training

Consider a distributed training job:



As N increases:

Required data throughput $\uparrow$

Eventually the storage system becomes the bottleneck.

Solutions include:

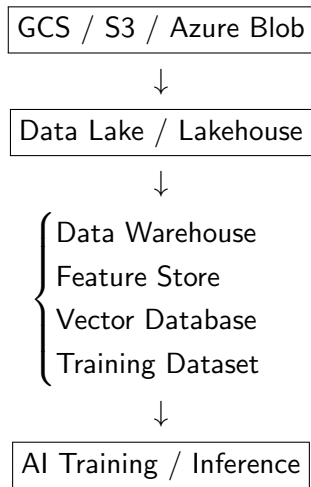
Storage for Real-Time AI

Training and inference have different requirements.

	Training	Inference
Data volume	Very large	Usually smaller
Latency	Less critical	Critical
Throughput	Critical	Important
Access	Sequential / batches	Random / online
Typical storage	Object storage	DB / cache / feature store

Production systems often combine several storage technologies.

Putting the Pieces Together



No single storage technology is optimal for every workload.

Choosing a Storage Solution

Ask these questions:

- ① What type of data are we storing?
- ② How large is it?
- ③ How frequently is it accessed?
- ④ Is access sequential or random?
- ⑤ What latency is required?
- ⑥ How many users / machines access it?
- ⑦ Does the data need transactions?
- ⑧ Does it need similarity search?
- ⑨ How important are versioning and lineage?
- ⑩ What are the storage and I/O costs?

Example: Recommendation System

Imagine an online recommendation system.

We might use:

- **Object storage** for raw interaction logs
- **Data lake** for historical datasets
- **Batch processing** for feature generation
- **Feature store** for user/product features
- **Vector database** for semantic similarity
- **Low-latency database/cache** for online requests

One AI system $\neq$ One database

Discussion: Design the Storage Architecture

Imagine your project has:

10 TB of raw data

containing:

- Millions of images
- Structured metadata
- User interaction logs
- Text descriptions
- Embeddings

You need:

- Large-scale model training
- Data analysis
- Real-time inference
- Semantic search

- **Object storage** is a foundation for large-scale AI data.
- **GCS, Amazon S3 and Azure Blob Storage** provide similar cloud object-storage capabilities.
- **Data lakes** allow organisations to retain large volumes of raw and processed data.
- **Lakehouses** add stronger table management and analytical capabilities on top of data-lake storage.
- **Vector databases** enable efficient similarity search over embeddings.
- **Feature stores** help manage features consistently between training and inference.
- **Metadata, lineage and versioning** are essential for trustworthy and reproducible AI.
- There is rarely a single optimal storage technology: **modern AI systems use a collection of complementary systems.**

Data infrastructure is an architectural problem.

The question is not:

“Which database should I use?”

It is:

“What access pattern, scale, latency and reliability requirements does my AI system have?”

Workload → Access Pattern → Storage Architecture